

SKILL

User Input

\$ARGUMENTS

You **MUST** consider the user input before proceeding (if not empty).

Pre-Execution Checks

Check for extension hooks (before tasks-to-issues conversion): - Check if .specify/extensions.yml exists in the project root. - If it exists, read it and look for entries under the hooks.before_taskstoissues key - If the YAML cannot be parsed or is invalid, skip hook checking silently and continue normally - Filter out hooks where enabled is explicitly false. Treat hooks without an enabled field as enabled by default. - For each remaining hook, do **not** attempt to interpret or evaluate hook condition expressions: - If the hook has no condition field, or it is null/empty, treat the hook as executable - If the hook defines a non-empty condition, skip the hook and leave condition evaluation to the HookExecutor implementation - When constructing slash commands from hook command names, replace dots (.) with hyphens (-). For example, speckit.git.commit → /skill:speckit-git-commit. - For each executable hook, output the following based on its optional flag: - **Optional hook** (optional: true): `` ` ##
Extension Hooks

```
**Optional Pre-Hook**: {extension}  
Command: `/{command}`  
Description: {description}
```

```
Prompt: {prompt}  
To execute: `/{command}`  
`` `
```

- **Mandatory hook** (optional: false):

```
## Extension Hooks
```

```
**Automatic Pre-Hook**: {extension}
```

```
Executing: `{command}`  
EXECUTE_COMMAND: {command}
```

Wait for the result of the hook command before proceeding to the Outline.

After emitting the block above you **MUST** actually invoke the hook and wait for it to finish before continuing. Run it the same way you would run the command yourself in this agent/session (the invocation may differ from the literal {command} id shown above, e.g. a skills-mode agent runs it as / skill:speckit-... or \$speckit-...). Emitting the block alone does not run the hook.

- If no hooks are registered or .specify/extensions.yml does not exist, skip silently

Outline

1. Run .specify/scripts/bash/check-prerequisites.sh --json --require-tasks --include-tasks from repo root and parse FEATURE_DIR and AVAILABLE_DOCS list. All paths must be absolute. For single quotes in args like "I'm Groot", use escape syntax: e.g 'I''m Groot' (or double-quote if possible: "I'm Groot").
2. **IF EXISTS:** Load .specify/memory/constitution.md for project principles and governance constraints.
3. From the executed script, extract the path to **tasks**.
4. Get the Git remote by running:

```
git config --get remote.origin.url
```

[!CAUTION] ONLY PROCEED TO NEXT STEPS IF THE
REMOTE IS A GITHUB URL

1. **Fetch existing issues for deduplication:** Before creating anything, build the set of task IDs you are about to process from tasks.md (each is a T followed by three digits, e.g. T001). Then use the GitHub MCP server's list_issues tool to look for issues that already cover those IDs. Do not pass a state value, since omitting it makes the tool return both open and closed issues. Request perPage: 100 to keep the number of calls down, and since the tool uses cursor-based pagination, request pages with the after parameter (using the endCursor from the previous response). For each issue title, match it against the task ID pattern \bT\d{3}\b (word boundaries so tokens like ST001 or T0010 are not matched by mistake; this also recognises titles written as T001 ..., T001: ... or [T001] ...) and, when it matches one of your task IDs, mark that ID as already having an issue. Stop paginating as soon as every task ID has been matched, or when there are no more pages, so you do not keep

fetching the whole repository's issue history once all task IDs are accounted for. This bounds the number of calls on repos with large issue histories and still prevents duplicates when the command is re-run after `tasks.md` is regenerated or the skill is re-invoked.

2. For each task in the list, use the GitHub MCP server to create a new issue in the repository that is representative of the Git remote. Task lines in `tasks.md` start with a markdown checkbox, so first strip the leading `- [ ]` (and any `[P]` / `[US#]` markers) to recover the task ID and its description. Create the issue with a single canonical title of the form `T001: <description>`, with the ID written once followed by the task description (for example, the line `- [ ] T001 Create project structure` becomes the title `T001: Create project structure`).
 - **Skip** any task whose ID is already present in the set of existing issues from the previous step, and report it (for example, `T001` already has an issue, skipping).
 - Only create issues for tasks that do not yet have a matching issue.

[!CAUTION] UNDER NO CIRCUMSTANCES EVER CREATE ISSUES IN REPOSITORIES THAT DO NOT MATCH THE REMOTE URL

Post-Execution Checks

Check for extension hooks (after tasks-to-issues

conversion): Check if `.specify/extensions.yml` exists in the project root. - If it exists, read it and look for entries under the `hooks.after_taskstoissues` key - If the YAML cannot be parsed or is invalid, skip hook checking silently and continue normally - Filter out hooks where `enabled` is explicitly `false`. Treat hooks without an `enabled` field as `enabled` by default. - For each remaining hook, do **not** attempt to interpret or evaluate hook condition expressions: - If the hook has no condition field, or it is null/empty, treat the hook as executable - If the hook defines a non-empty condition, skip the hook and leave condition evaluation to the `HookExecutor` implementation - When constructing slash commands from hook command names, replace dots (`.`) with hyphens (`-`). For example, `speckit.git.commit` → `/skill:speckit-git-commit`. - For each executable hook, output the following based on its optional flag: - **Optional hook** (optional: true): `` ` ## Extension Hooks

****Optional Hook****: {extension}

Command: `{command}`

Description: {description}

Prompt: {prompt}

To execute: `{command}`
`{command}`

- **Mandatory hook** (optional: false):

Extension Hooks

```
**Automatic Hook**: {extension}  
Executing: `{command}`  
EXECUTE_COMMAND: {command}
```

After emitting the block above you **MUST** actually invoke the hook and wait for it to finish before continuing. Run it the same way you would run the command yourself in this agent/session (the invocation may differ from the literal {command} id shown above, e.g. a skills-mode agent runs it as /skill:speckit-... or \$speckit-...). Emitting the block alone does not run the hook.

- If no hooks are registered or .specify/extensions.yml does not exist, skip silently